Interview Preparation Guide

Narrative Engine — ML/AI Engineer Assignment

Part 1: Topic Evolution Pipeline | Part 2: Semantic Topic Search

EASY Conceptual / Definitions	**MEDIUM** Applied / Design Decisions
HARD Trade-offs / System Design	**FOLLOW-UP** Interviewer Challenges

PART 1 — Topic Evolution Pipeline

1. UNDERSTANDING & FUNDAMENTALS

Q1: **[EASY]** What is the Jaccard similarity score and why did you use it as the primary metric for comparing topics?

Ideal Answer:

Jaccard similarity measures the overlap between two sets — specifically, the intersection divided by the union of those sets.

I used it as the primary metric because the core problem is: "which posts appear in both topics?" Each topic is essentially a set of post IDs, and Jaccard directly quantifies how much the same real-world content is being grouped together. It is:

- Symmetric — topic A vs B equals topic B vs A
- Naturally bounded between 0 and 1
- Independent of topic size — a small stable topic with 10 posts scores correctly against a large one
- Interpretable — Jaccard 0.8 literally means 80% of content is shared

The key insight is that post IDs are ground truth: the same post re-appearing in two runs is unambiguous evidence that two topics are related, regardless of how titles or summaries are phrased.

Follow-up: If two topics have a Jaccard score of 0.05 but both discuss the exact same event — how would you detect that?

Q2: **[EASY]** Explain what TF-IDF is and why you applied it here on top of Jaccard similarity.

Ideal Answer:

TF-IDF (Term Frequency–Inverse Document Frequency) is a text vectorization technique that weights words by how frequently they appear in a document relative to how common they are across all documents. Rare, discriminating words get high scores; common stopwords get near zero.

In this solution, TF-IDF + cosine similarity is used as a secondary signal alongside Jaccard. The reason is that Jaccard alone is post-overlap-based — it does not capture semantic similarity in the topic titles or summaries. TF-IDF helps in cases like:

- Two topics that share few overlapping posts but discuss the same subject
- Topics where the NE model re-assigned posts slightly differently between runs

Together they form a multi-signal approach: Jaccard gives content-level signal, TF-IDF+cosine gives metadata-level signal. Both are recorded in the output, giving analysts two lenses on similarity.

Follow-up: Why not use TF-IDF as the primary signal instead of Jaccard?

Q3: **[EASY]** What is cosine similarity and how does it differ from Jaccard in this context?

Ideal Answer:

Cosine similarity measures the angle between two vectors in a high-dimensional space. Two documents are similar if their TF-IDF vectors point in the same direction, regardless of their magnitude (length).

Key differences in this context:

- Jaccard operates on sets of post IDs — it measures literal content overlap
- Cosine similarity operates on text vectors (titles + summaries) — it measures semantic/linguistic similarity
- Jaccard is deterministic given the post assignments; cosine varies with vocabulary choices in the AI-generated titles/summaries

Cosine similarity is more susceptible to noise from AI-generated text (synonyms, phrasing variation), which is why Jaccard is the primary metric for categorization.

Q4: **[MEDIUM]** Walk me through your categorize_evolution() function. Why did you choose these specific Jaccard thresholds?

Ideal Answer:

The function maps a Jaccard score to a human-readable label for stakeholders:

- Stable (≥ 0.8): The vast majority of posts are shared. The topic is essentially unchanged — perhaps minor additions or edge post re-assignments.
- Evolved (0.5 – 0.8): A clear majority of posts overlap, but there has been meaningful change — new angles added or older content dropped. This is the "same story, new chapter" category.
- Fragmented (0.1 – 0.5): Some posts are shared but not the majority. The topic may be splitting, merging, or diverging significantly.
- Dropped/Noise (< 0.1): Almost no post overlap. This pair is likely noise from the pipeline generating spurious connections, or the topic has genuinely died.

The thresholds are heuristics — they are reasonable starting points based on general NLP practice but should be validated against ground truth labels. The right thresholds are domain-specific.

Follow-up: What would happen to your categorization if the Narrative Engine ran on dramatically different-sized windows?

2. DESIGN DECISIONS

Q5: **[MEDIUM]** Why did you choose post-overlap (Jaccard on post IDs) as your primary matching signal rather than purely comparing topic titles or summaries?

Ideal Answer:

This is the central design insight: post IDs are ground truth, AI-generated text is not.

Three reasons:

- Titles and summaries are generated by the Narrative Engine — they may use different phrasings, synonyms, or levels of detail across runs even for identical topics.
- Post IDs are deterministic. If the same post ID appears in two topics across two runs, that is an objective signal, not a model-dependent artifact.
- This also respects the constraint that we cannot use an LLM to compare topics — text-based LLM comparison would be higher quality but is explicitly disallowed.

The combination of both signals (Jaccard + cosine) provides a richer picture, but the post-overlap signal drives the primary categorization decision.

Follow-up: What if the data pipeline had an ID generation bug and some posts had duplicate IDs across runs?

Q6: **[HARD]** How does your pipeline detect a topic SPLIT (one topic becoming two) and a MERGE (two topics becoming one)? Is it explicit or implicit?

Ideal Answer:

It is implicit, not explicit — which is an important limitation to acknowledge.

The run_comparison method generates ALL pairs of (prior_topic, later_topic) that share at least one post. This means:

- Split detection: If Topic A in Run 1 maps to BOTH Topic B and Topic C in Run 2 with moderate Jaccard scores (e.g. 0.3 and 0.4), the output will contain two rows for Topic A — this is the signature of a split.
- Merge detection: If Topic B and Topic C in Run 1 both map to Topic A in Run 2, that is a merge signature — Topic A appears twice as the "New Topic" column.

The output DataFrame does not have a dedicated "Split" or "Merge" column — analysts would need to detect these patterns post-hoc by grouping on Earlier Topic or New Topic and looking for one-to-many mappings.

An explicit improvement would be to add a post-processing step that: (1) identifies topics with multiple high-overlap partners, (2) computes the partition of posts, (3) labels rows explicitly as SPLIT or MERGE.

Follow-up: How would you handle the case where a topic splits and one half merges with something else simultaneously?

Q7: **[MEDIUM]** You merge three post DataFrames on post ID with an outer join, then filter for posts appearing in at least 2 runs. What is the reasoning behind this design?

Ideal Answer:

The outer join is essential because the three runs use overlapping but not identical date windows. Not every post will appear in all three runs:

- A post from Jan 20 may appear in Run 1 and Run 2 but not Run 3 (which starts Jan 22).
- Dropping posts that do not appear in all three runs would create a massive false negative problem — we would miss valid topic continuations.

The ≥2 filter serves a specific purpose: we only care about posts that create a bridge between at least two runs. A post appearing in only one run cannot inform any between-run comparison — it would only add noise to the similarity computation.

The fillna(-1) followed by replace(-1, "-1") ensures that missing topic assignments are handled as valid string values and do not create NaN-based join artifacts.

Follow-up: What happens to your metrics if a post appears in Run 1 and Run 3 but NOT Run 2?

3. ALTERNATIVE APPROACHES

Q8: **[HARD]** What alternative approaches could you have used to compare topics, and why did you not choose them?

Ideal Answer:

BERTopic / Sentence Embeddings on posts: You could embed all posts and compare topic centroids across runs. More semantically rich, but significantly more compute-intensive, harder to

interpret, and potentially introduces drift in embedding space. Also harder to justify "same topic" claims to stakeholders.

Exact entity matching: Compare topics purely by key_entities. Fast and interpretable, but brittle — the NER model may extract different surface forms or miss entities across runs, causing false negatives.

Graph-based topic tracking: Build a bipartite graph of (run1_topics) — (run2_topics) connected by Jaccard weight, then apply community detection. More principled for multi-topic merges/splits, but adds dependency on graph libraries and complexity.

LDA/NMF re-alignment: Run a global topic model across all three runs and re-align local topics to global ones. Theoretically principled but violates the independence constraint of the runs and adds complexity.

The chosen approach (Jaccard on post IDs + TF-IDF cosine) is the best balance of interpretability, correctness, and computational efficiency for this specific data structure.

🔁 **Follow-up:** In what scenario would the graph-based approach outperform yours significantly?

Q9: **[MEDIUM]** Could you have used Overlap Coefficient instead of Jaccard? When would it be better?

✅ **Ideal Answer:**

Yes — Overlap Coefficient = |A ∩ B| / min(|A|, |B|). It measures overlap relative to the SMALLER set, making it insensitive to size asymmetry.

When it would be better: When one topic is a strict subset of another — for example, a large "AI News" topic in Run 1 that splits into "AI Regulation" and "AI Research" in Run 2. Jaccard would penalize the smaller child topics because the union is large. Overlap Coefficient would correctly show high similarity between parent and each child.

The trade-off: Overlap Coefficient can be misleadingly high when a tiny topic happens to be entirely contained in a large one, even if the large topic has moved on completely. Jaccard is more conservative and penalizes this, which is the correct behavior for detecting stable topics.

4. EDGE CASES & FAILURE SCENARIOS

Q10: **[MEDIUM]** How did you handle the topic -1 (outlier) posts in your pipeline? What edge cases does this create?

✅ **Ideal Answer:**

Topic -1 is stored as the string "-1" after the int conversion and fillna logic. This means all outlier posts are grouped under a single "super-topic" labeled "-1" in the comparison.

The critical edge case: If many posts are outliers in Run 1 but the same posts get assigned to real topics in Run 2, the pipeline will compute a Jaccard score between topic -1 (Run 1) and real topics (Run 2). This would make topic -1 appear to have many "successors", which is semantically meaningless.

Better handling: Filter out all rows where run_1, run_2, or run_3 == "-1" before running comparisons. Topic -1 should be treated as a null/unknown category, not a real topic. The current code does not do this filtering explicitly, which is a bug.

🔁 **Follow-up:** How many posts in the dataset are outliers, and what percentage does that represent? How would you surface this to stakeholders?

Q11: **[HARD]** Your pipeline processes only posts that appear in at least 2 runs. What topics or changes might you be systematically missing?

Ideal Answer:

Several categories of topics will be invisible to the pipeline:

- New topics that emerge in Run 3 only: If all posts in a Run 3 topic are new posts (post-dating Run 1 and Run 2), none of those posts appear in the merged DataFrame, so the topic has no representation.
- Dying topics: A topic in Run 1 whose posts do not recur in Run 2 or 3 (perhaps it was a short news spike) will be invisible. It looks like it was never there.
- Topics from posts near date boundaries: Posts at the edge of the overlap window are more likely to appear in only one run.

To fix this: After the main pipeline, add a separate pass that identifies topics in each run that have zero matches in the merged dataset, and label them as "Emerging" (new topics) or "Expired" (topics that disappeared).

Follow-up: How would you detect if a topic completely disappeared between Run 2 and Run 3?

Q12: **[MEDIUM]** What happens to your pipeline if the Narrative Engine produces a very large "catch-all" topic containing thousands of posts?

Ideal Answer:

A large catch-all topic (e.g., a general "Politics" topic with 3,000 posts) would cause Jaccard inflation issues:

- It will match with nearly every other topic in the other runs because its post set is so large that intersections are common.
- But each individual Jaccard score will be low because the union is dominated by the large topic's massive post set.
- This creates many low-Jaccard "Fragmented" pairings in the output that are not meaningful.

Mitigation strategies:

- Filter out topics above a size threshold before comparison (or flag them separately).
- Use a quality score (e.g., topic coherence from the NE) to down-weight or exclude poorly-performing generalist topics.
- Apply a minimum Jaccard threshold before including a pair in results (the current code does not have a minimum threshold beyond intersection > 0).

5. OPTIMIZATION & SCALABILITY

Q13: **[HARD]** What is the time complexity of your run_comparison method? How would it scale to millions of topics?

Ideal Answer:

The current complexity is $O(T_1 \times T_2 \times P)$ in the worst case, where T_1 and T_2 are topic counts per run and P is the average posts per topic.

The inner loop iterates over all pairs of (prior_topic, later_topic). With 442 × 271 = ~120,000 pairs, this is manageable. But at millions of topics, this becomes infeasible.

Scalability improvements:

- Inverted index: Build a map of post_id → topic_id. For each prior topic, look up only the later topics that share at least one post. This reduces the effective pairs dramatically.
- MinHash / LSH: Approximate Jaccard similarity at scale using MinHash and Locality-Sensitive Hashing. Reduces to O(T × b × r) where b×r are band and row parameters.
- Spark / distributed computing: Shard topics by post ID hash ranges and compute similarities in parallel.
- Threshold early exit: Skip pairs where intersection is very small relative to sizes (Jaccard upper bound check without computing full union).

🔁 **Follow-up:** What is the Jaccard upper bound trick and how would you implement it?

Q14: **[MEDIUM]** Your TF-IDF vectorizer is re-fit for every pair of topics in the inner loop. What is the problem with this and how would you fix it?

✅ **Ideal Answer:**

This is a significant inefficiency bug. The vectorizer.fit_transform([text_p, text_l]) call inside the loop re-fits the vocabulary and computes IDF weights from scratch for every single topic pair. This means:

- IDF weights are computed from only 2 documents — this is statistically meaningless (IDF on 2 docs is essentially just binary).
- The vocabulary changes per pair, making cosine scores non-comparable across pairs.
- Performance: $O(T_1 \times T_2)$ vectorizer fits instead of O(1).

The correct approach:

- Fit the vectorizer ONCE on the entire corpus of all topic texts from both runs.
- Transform all topic texts in a batch to get a full matrix.
- Compute the full cosine similarity matrix at once using sklearn's cosine_similarity on the full matrices.

This reduces from $O(T^2)$ vectorizer fits to 1 fit and 1 matrix multiplication.

🔁 **Follow-up:** After fixing this, what would the correct interpretation of cosine similarity be that you could not claim before?

6. CODE-LEVEL DEEP DIVE

Q15: **[MEDIUM]** In get_entity_changes(), you extract only entity_list[0] from each entity cluster. What is the assumption here and when could it be wrong?

✅ **Ideal Answer:**

The data format for entities is a list of lists: [[all variations of entity 1], [all variations of entity 2], ...]. Index [0] is assumed to be the canonical/primary variant of each entity.

This assumption could be wrong when:

- The NER model stores variants in arbitrary order — e.g., ["Trump", "Donald Trump", "President Trump"] vs ["Donald Trump", "Trump"] across runs. The canonical form differs, causing "Trump" and "Donald Trump" to appear as separate entities in the diff even though they are the same.
- The entity cluster is empty (edge case not handled) — entity_list[0] would raise an IndexError.

Better approach: Normalize entities by choosing the longest or most frequent variant as canonical, or compare entire sets of variants across all entity clusters (set intersection on all variants, not just index 0).

Q16: **[MEDIUM]** In extract_data(), you do str(int(post["topic"])). What edge cases could cause this to fail or produce incorrect results?

Ideal Answer:

Several edge cases:

- If topic is a float like 3.0, int(3.0) = 3, which is fine. But if it is NaN (from a missing field), int(NaN) throws a ValueError.
- If topic is a string like "3a" or an empty string, int() will throw a ValueError.
- If topic is None (missing key + .get() default), the code will throw a TypeError since int(None) is invalid.
- The data description explicitly says "This is messy real world data with many edge cases" — so these failures are likely.

A safer pattern: use a try/except or pd.to_numeric() with errors="coerce", followed by fillna(-1) to handle all non-numeric topic IDs gracefully.

Follow-up: How would you add data validation / schema checking at the point of ingestion?

7. VALIDATION STRATEGY

Q17: **[HARD]** How would you validate that your topic evolution pipeline is producing correct results? Walk me through a full validation strategy.

Ideal Answer:

A rigorous validation strategy would have three layers:

Layer 1 — Human annotation (gold standard): Manually annotate a sample of ~100–200 topic pairs as "same", "evolved", "split", "merged", or "new/dead". Compute precision, recall, and F1 for each category against pipeline output.

Layer 2 — Threshold sensitivity analysis: Run the pipeline with varying Jaccard thresholds (0.5, 0.6, 0.7, 0.8, 0.9) and measure how category distributions change. Stable distributions across a range suggest robust thresholds.

Layer 3 — Sanity checks on known events: The data covers news from late January to late February 2026. Identify 2–3 major news events known to have evolved (e.g., a story that escalated). Verify the pipeline correctly marks those as "Evolved" or "Fragmented".

Automated regression tests: Create synthetic datasets with known ground truth (3 posts per topic, guaranteed overlap patterns) and assert pipeline output matches expected labels exactly.

Follow-up: What metrics would you use to measure the quality of the categorization beyond accuracy?

PART 2 — Semantic Topic Search

1. UNDERSTANDING & FUNDAMENTALS

Q18: **[EASY]** What is a sentence transformer and why did you choose multi-qa-MiniLM-L6-cos-v1 specifically over all-MiniLM-L6-v2?

Ideal Answer:

A sentence transformer is a BERT-variant fine-tuned with a siamese/triplet network objective to produce semantically meaningful sentence-level embeddings — i.e., sentences with similar meaning have similar vectors in the embedding space.

all-MiniLM-L6-v2 is a general-purpose sentence similarity model. multi-qa-MiniLM-L6-cos-v1 is specifically fine-tuned for asymmetric retrieval tasks: encoding a query against a set of passages/documents.

This distinction is crucial here: the user's query is structurally very different from the topic "documents" (which are title+summary+entities). The multi-qa model is optimized for this asymmetric scenario — it produces better query-document matching than a symmetric model.

The comment in the code (showing both options) suggests this was a deliberate, tested decision — a good demonstration of model selection awareness.

Follow-up: How would you evaluate whether this model selection was actually correct for this use case?

Q19: **[EASY]** Explain what the deduplication step does in compare_topics() and why it is necessary before embedding.

Ideal Answer:

The deduplication step uses Jaccard similarity on token sets to identify topics that are likely duplicates across the three runs — the same real-world topic appearing in multiple files.

Why it is necessary:

- Without deduplication, the same topic would appear 2–3 times in the embedding index (once per run). A search would return near-duplicate results occupying multiple top-5 slots, providing the user with less diversity of information.
- It reduces the embedding index size, improving search latency and memory usage.
- It ensures that a single query does not surface the "Groundhog Day" topic from all three runs simultaneously — the user gets unique topics.

The Jaccard threshold of 0.5 is a design choice — lower values deduplicate more aggressively, potentially merging distinct but related topics.

Follow-up: What would a Jaccard threshold of 0.1 do to your search results? What about 0.95?

2. DESIGN DECISIONS

Q20: **[HARD]** You are not allowed to use an LLM to check every topic for similarity to the query. Why is embedding + cosine search a valid alternative? What does it sacrifice?

Ideal Answer:

Embedding + cosine search works because the sentence transformer creates a shared vector space where semantically similar texts have similar directions. A query about "Super Bowl political controversy" and a topic about "Kendrick Lamar's halftime show political protest" should have nearby vectors.

What it gains over LLM per-topic comparison:

- Speed: O(1) embedding of query + O(T) cosine operations vs O(T) LLM API calls. For millions of topics, this is the difference between milliseconds and hours.
- Cost: A single embedding call vs T inference calls at full LLM cost.

What it sacrifices:

- Reasoning: An LLM can understand nuanced user intent like "I want political controversy, NOT sports analysis." Embeddings encode both dimensions but cannot reason about which dimension is more important.
- Context: LLMs can follow multi-step reasoning ("show me topics that would be interesting to a researcher studying media bias"). Embeddings do not reason.
- Negation: "I do NOT want X" is poorly handled by cosine similarity — the embedding of "not violent" and "violent" can be close.

Follow-up: How would you handle a user query that contains negation — e.g., "I want protest topics but NOT violent ones"?

Q21: **[MEDIUM]** Your combination string is: "Title: {title}; Summary: {summary}; entities: {entities}". Why did you format it this way and what are the trade-offs of this design?

Ideal Answer:

The structured format serves two purposes:

- Prefix labels (Title:, Summary:, entities:) help the transformer understand which part of the text is which — though modern transformers largely ignore this without specific fine-tuning.
- Including all three signals (title, summary, entities) in one string ensures the embedding captures the full semantic profile of the topic — not just its headline or just its named entities.

Trade-offs:

- Entities are extracted as primary variants only — if the topic is about "Donald Trump" but the entity list reads "Trump", a query for "Donald Trump" might miss it.
- Summaries can be noisy or hallucinated by the NE model — injecting low-quality text degrades the embedding.
- Long combination strings may exceed the model's context window (128 tokens for MiniLM), causing truncation and loss of information.
- Entity weighting: Entities are equally weighted with all other text. But for proper-noun-heavy queries, entities are more discriminating and should be up-weighted.

Follow-up: How would you handle topics with very long summaries that exceed the model token limit?

3. ALTERNATIVE APPROACHES

Q22: **[HARD]** What is BM25 and how does it compare to your semantic embedding approach for this search problem?

Ideal Answer:

BM25 (Best Match 25) is a probabilistic keyword-based ranking function — an evolution of TF-IDF that accounts for document length normalization and term saturation. It is the backbone of Elasticsearch and many production search systems.

BM25 vs Semantic Embeddings for this use case:

- BM25 requires keyword overlap — the user query must share words with the topic text. "Super Bowl halftime controversy" would miss a topic titled "Performers face political backlash at NFL championship event".
- Semantic embeddings handle paraphrase/synonym matching — "halftime show" and "halftime performance" map to similar vectors.
- BM25 is much faster and has no embedding overhead — ideal for very large corpora.
- BM25 handles exact entity names better (e.g., "Kendrick Lamar") where embeddings may lose specificity.

Best production approach: Hybrid BM25 + semantic embedding (Reciprocal Rank Fusion or cross-encoder re-ranking). BM25 for recall, embeddings for precision.

Follow-up: How would you implement a hybrid BM25 + embedding system? What would the architecture look like?

Q23: **[MEDIUM]** Could you have used FAISS or another ANN library for the search instead of brute-force cosine similarity? When would that matter?

Ideal Answer:

Yes. The current implementation computes exact cosine similarity between the query vector and ALL topic embeddings — O(T × d) where d is embedding dimension (384 for MiniLM).

For this problem (hundreds to low thousands of topics), brute-force is perfectly fine — it runs in microseconds. FAISS would provide no practical benefit at this scale.

FAISS / ANN becomes necessary when:

- T > 100,000 topics — brute-force becomes too slow for interactive search (> 100ms latency).
- Running millions of queries per day — even O(T) becomes significant under high QPS.

FAISS (HNSW or IVF index) trades a small accuracy loss (approximate nearest neighbors vs exact) for 10–1000x speedup. For a search application where top-5 relevance is sufficient, this is an acceptable trade-off.

4. EDGE CASES & FAILURE SCENARIOS

Q24: **[HARD]** A user submits the query: "I want to see topics about the recent violence during protests." How might your pipeline fail or produce poor results?

Ideal Answer:

Several failure modes for this specific query:

- Dual topic sensitivity: "violence" and "protests" are both heavy anchors in the embedding. If there are topics about protests without violence, or violence without protests, they may score almost as high as truly matching topics.
- Recency blindness: The query says "recent" — but all three runs are from the same February 2026 window. The system has no ability to prioritize more recent topics over older ones within the dataset.

- Emotional register: "I am interested in seeing what emotions are expressed" — this intent is invisible to keyword-based and pure semantic embedding approaches. The embedding of the combination string does not capture sentiment or emotional register.
- Missing context: If the protest topics are only described with neutral entity names (e.g., "Gaza protests") and no violence-specific vocabulary, the cosine similarity may miss them entirely.

Improvement: Use an LLM to expand the query into a richer semantic document before embedding (HyDE — Hypothetical Document Embeddings).

🔄 **Follow-up:** What is HyDE and how would you implement it as an enhancement here?

Q25: **[MEDIUM]** Your deduplication uses a Jaccard threshold of 0.5. What happens if two genuinely different topics happen to share 50%+ of their vocabulary?

✅ **Ideal Answer:**

This creates a false duplicate — a unique topic gets dropped from the embedding index. It would then be permanently invisible to any search query.

When this is likely to occur:

- Short topics with small token sets — even 3–4 shared words can push Jaccard above 0.5 for short texts.
- Topics that share a common theme but diverge in focus — e.g., "Super Bowl TV ratings" vs "Super Bowl halftime TV broadcast controversy" share many tokens.

Improvements:

- Use TF-IDF weighted Jaccard (or WMD) instead of raw token Jaccard — this down-weights common/generic words.
- Apply deduplication at the embedding level instead — two topics are duplicates if their sentence embeddings have cosine similarity > 0.95 (much stricter semantic identity test).
- Instead of dropping duplicates, merge them by averaging their embeddings — preserves information while reducing redundancy.

5. OPTIMIZATION & SCALABILITY

Q26: **[HARD]** How would you productionize this search system to serve real-time queries at scale? Walk me through the architecture.

✅ **Ideal Answer:**

A production architecture would separate the offline indexing pipeline from the online serving layer:

Offline pipeline: New run outputs trigger a batch process that: (1) deduplicates new topics against the existing index, (2) generates embeddings in batch on GPU, (3) upserts vectors into a vector database (Pinecone, Weaviate, or pgvector).

Online serving: User query → LLM query expansion (optional HyDE) → embed query → ANN lookup in vector DB → return top-K results with metadata. Target latency: < 200ms.

Monitoring: Log query embeddings, result IDs, and user click-through rates. Use click-through as implicit relevance signal. Monitor embedding drift — if the sentence transformer is updated, re-embed the full index.

Caching: Cache query embeddings for repeated queries (Redis with TTL). Pre-compute embeddings for known common query patterns.

A/B testing: Run experiments with different models (BM25 vs embedding vs hybrid) using interleaved ranking or shadow mode evaluation.

🔁 **Follow-up:** How would you handle a situation where the embedding model needs to be swapped out for a better one? What is the migration path?

6. REAL-WORLD & CROSS-CUTTING QUESTIONS

Q27: **[HARD]** Across both parts, you had to process "messy real-world data." How did you approach data quality and what systematic problems might remain?

✅ **Ideal Answer:**

Handled defensively in code:

- str(int(x)) pattern converts floats/ints to strings uniformly.
- isinstance() checks on summary (list vs string) handle format inconsistencies.
- fillna(-1) / replace(-1, "-1") handles missing join values.
- Topic -1 filtering for outliers.

Likely remaining issues:

- Malformed JSON — no validation at load time; a truncated file would crash the pipeline silently after partial processing.
- Unicode/encoding issues in post text — social media data is rich in emojis, non-ASCII characters, URLs, which can break TF-IDF tokenization.
- Duplicate post IDs — the problem statement notes IDs are random hashes. Hash collisions or bugs could create false cross-topic bridges.
- None entities — the problem statement explicitly mentions some outlier posts have None entities. Code accesses entities without guarding against None in all paths.

🔁 **Follow-up:** What is the first data quality check you would add before any processing begins?

Q28: **[MEDIUM]** If a stakeholder asked you to explain WHY two topics were matched as "Evolved," what information would you surface from your pipeline?

✅ **Ideal Answer:**

The pipeline already provides the right building blocks for explainability:

- Jaccard Score: "X% of the posts discussing this topic in Run 1 reappeared in the Run 2 topic."
- Cosine Similarity: "The titles and summaries of both topics are Y% textually similar."
- Added Entities: "New key entities in the evolved topic: [list]."
- Removed Entities: "Entities that were important but are no longer prominent: [list]."
- Earlier Title vs New Title: Side-by-side comparison of how the narrative framing changed.

For a richer explanation, you could also surface: (1) the posts that are new to the later topic (not in the prior), (2) the posts that dropped out, and (3) a sample of the actual text for each group. This gives stakeholders a story, not just numbers.

Q29: **[HARD]** Looking across both solutions holistically — what is the single biggest weakness, and what would your next iteration look like?

✅ **Ideal Answer:**

The biggest weakness is the lack of an end-to-end quality signal. Neither part has a mechanism to measure whether the outputs are actually correct — both rely entirely on heuristics that were never validated against ground truth.

Part 1's Jaccard thresholds are arbitrary. Part 2's search results are never evaluated for relevance. Without ground truth, we cannot know if these systems are working or just producing plausible-looking output.

Next iteration priorities:

- Part 1: Build a small annotation tool to label 200 topic pairs. Use these to calibrate Jaccard thresholds, fix the topic -1 handling, and measure F1 per category. Add explicit SPLIT/MERGE detection.
- Part 2: Run 20–30 example queries with manual relevance judgements. Compute nDCG@5. Test HyDE for query expansion. Implement hybrid BM25+embedding. Fix the TF-IDF vectorizer bug in Part 1 as a side effect of shared refactoring.
- Both: Add a data validation layer at ingestion, structured logging for observability, and unit tests for edge cases.

Follow-up: If you had one week to productionize just one of the two parts, which would you choose and why?

FOLLOW-UP QUESTIONS — Answered

Each follow-up question from the main guide is answered below in full. These are the harder, probing questions an interviewer would use to challenge your thinking after your initial answer.

PART 1 FOLLOW-UPS

QFU-1: **[HARD]** If two topics have a Jaccard score of 0.05 but both discuss the exact same event — how would you detect that? (Follow-up to Q1)

✅ Ideal Answer:

This is a real failure mode of pure post-overlap matching: two topics can cover the same event but be populated with completely different posts — for example, because the NE model re-clustered posts differently in the two runs, or because the overlapping date window shifted and brought in new posts about the same event.

Detection strategies:

- Cosine similarity on combined text: The TF-IDF cosine score should be high even if Jaccard is low, because the titles and summaries would reference the same event vocabulary. A high cosine + low Jaccard is the signature of "same event, different post assignment".
- Entity overlap: Compare key_entities across the two topics. If both topics share 3+ key entities (e.g., ["Groundhog Day", "Punxsutawney Phil", "Pennsylvania"]), they are almost certainly covering the same event.
- Title embedding similarity: Use a sentence transformer to embed the titles alone — this is fast and captures semantic identity even with different wording.

The fix: Promote cosine similarity to a co-equal signal. If cosine > 0.85 AND entity overlap > 50%, classify as "Same Topic — Different Posts" even when Jaccard is below the Fragmented threshold. This requires defining a new category in categorize_evolution().

QFU-2: **[MEDIUM]** What would happen to your categorization if the Narrative Engine ran on dramatically different-sized windows? (Follow-up to Q4)

✅ Ideal Answer:

Jaccard is sensitive to window size because the denominator (union) grows with the number of posts in each topic. If Run 1 uses a 30-day window and a hypothetical Run 4 uses a 7-day window, topics in Run 4 would have far fewer posts. The Jaccard between a large Run 1 topic and a small Run 4 topic would be artificially low even if the small topic is a perfect subset of the large one.

Concretely: A stable topic with 200 posts in Run 1 and 40 posts in Run 4 (all 40 overlapping) would score Jaccard = 40/200 = 0.2 — classified as "Fragmented", which is wrong.

Mitigations:

- Use Overlap Coefficient instead of Jaccard for cross-window comparisons: 40/min(200,40) = 1.0 — correctly Stable.
- Normalize Jaccard by window size ratio — penalize expected difference due to window mismatch.
- Maintain consistent window sizes across all runs (the current data does this — all runs are 30-day windows — so this is a robustness concern for future use).

QFU-3: **[HARD]** What if the data pipeline had an ID generation bug and some posts had duplicate IDs across runs? (Follow-up to Q5)

Ideal Answer:

This would silently corrupt every metric in the pipeline. Duplicate IDs across runs would create false post-overlap bridges between unrelated topics — a post ID that appears in "Groundhog Day" in Run 1 and (due to a collision) in "Super Bowl Coverage" in Run 2 would make those two topics appear related with a non-zero Jaccard score.

Detection:

- Run a cardinality check: count distinct post IDs in each file. If the count of unique IDs across all three files is significantly less than the sum of IDs per file, there are collisions.
- Cross-file ID check: query for IDs that appear in multiple files but whose post text is completely different — a collision signature.

Fix: Namespace IDs by run — prefix all IDs with the run date (e.g., "20260219_abc123") before merging. This makes cross-run collisions impossible and is a defensive design that costs nothing.

This is a great example of why post IDs as ground truth require a data integrity assumption — one that should be explicitly tested, not assumed.

QFU-4: **[HARD]** How would you handle the case where a topic splits and one half merges with something else simultaneously? (Follow-up to Q6)

Ideal Answer:

This is a complex topology: Topic A in Run 1 → {Topic B, Topic C} in Run 2, but then Topic C + Topic D in Run 2 → Topic E in Run 3. The current pipeline would surface this as two separate observations but has no way to link them into a coherent narrative.

A graph-based approach handles this naturally:

- Represent all topics across all runs as nodes. Add weighted edges between (Run i topic, Run i+1 topic) where weight = Jaccard score.
- Apply a minimum edge weight threshold (e.g., 0.1) to prune noise.
- Each connected component in the resulting graph represents a "topic lineage" — a family of related topics across runs.
- Within each component, node in-degree > 1 = merge; out-degree > 1 = split. Complex cases (split-then-merge) are just paths through the graph.

NetworkX could implement this in ~20 lines. The key insight is that the pairwise DataFrame already contains all the information needed — it just needs to be recast as a graph structure.

QFU-5: **[MEDIUM]** What happens to your metrics if a post appears in Run 1 and Run 3 but NOT Run 2? (Follow-up to Q7)

Ideal Answer:

This post passes the ≥2 runs filter and is included in the merged DataFrame. However, its contribution to metrics is asymmetric:

- Run 1→2 comparison: run_2 is -1 (filled as outlier/missing). The post contributes to the Run 1 topic's post set but NOT to any Run 2 topic set. This slightly inflates the Run 1 topic's union size, marginally deflating Jaccard for Run 1→2.
- Run 2→3 comparison: run_2 is -1, so the post contributes to topic "-1" in Run 2 and to a real topic in Run 3. This creates a spurious connection between topic -1 (Run 2) and a real topic (Run 3) — which is the same topic -1 problem discussed in Q10.

The cleanest fix: for any hop comparison (Run i → Run i+1), only include posts that have valid (non -1) topic assignments in BOTH runs being compared. Filter per-hop rather than globally.

QFU-6: **[HARD]** In what scenario would the graph-based approach outperform yours significantly? (Follow-up to Q8)

✅ Ideal Answer:

The graph-based approach would significantly outperform the current pairwise DataFrame approach in three scenarios:

- Complex multi-hop splits and merges: When a topic in Run 1 fragments into 4 sub-topics in Run 2 which then re-consolidate into 2 topics in Run 3, the graph captures the full lineage. The current approach only shows individual pairwise connections — the analyst has to mentally reconstruct the lineage.
- Large-scale analysis with many runs: If the NE runs daily for 90 days (90 runs), a pairwise DataFrame approach produces 89 separate comparison tables. A graph trivially spans all 90 runs as a temporal DAG, enabling queries like "which topics survived all 90 days?"
- Community detection for topic families: Graph algorithms like Louvain or HITS can identify clusters of related topics across all runs simultaneously — things like "this whole family of topics is about the Ukraine war" — which the pairwise approach cannot express.

The trade-off is interpretability and engineering complexity — graphs require more infrastructure and are harder to explain to non-technical stakeholders than a simple CSV.

QFU-7: **[MEDIUM]** How many posts in the dataset are outliers (topic -1), and how would you surface this to stakeholders? (Follow-up to Q10)

✅ Ideal Answer:

Without running the data: based on typical NE/BERTopic model behavior, outlier rates of 10–30% are common on noisy social media data. A high outlier rate (>40%) is a signal that the model is poorly calibrated for this domain.

To compute it: after extract_data(), filter the post DataFrame for rows where topic == "-1" and divide by total post count per run. Compare across all three runs — if the outlier rate is rising between runs, the model's clustering quality may be degrading.

Surfacing to stakeholders:

- Include an "Outlier Rate" metric in the pipeline summary output — one number per run.
- Flag if the rate exceeds a threshold (e.g., >25%) as a data quality warning.
- In dashboards, show outlier posts as a special "-1 cluster" so analysts can manually review them — sometimes outliers cluster around emerging topics that the model hasn't yet learned.

QFU-8: **[MEDIUM]** How would you detect if a topic completely disappeared between Run 2 and Run 3? (Follow-up to Q11)

✅ Ideal Answer:

A topic has "disappeared" if it has no successors — i.e., no Run 3 topic shares even a single post with it.

Detection algorithm:

- Get the set of all Run 2 topic IDs that appear as "Earlier Topic" in the evolution_2_to_3 DataFrame.
- Get the set of all unique Run 2 topic IDs from the topic metadata file.
- Topics in the metadata set but NOT in the evolution DataFrame are "Expired" — they had no post-overlap with any Run 3 topic.

Nuance: A topic may disappear from the pipeline not because it truly died, but because all its posts were date-boundary posts that no longer appear in Run 3's window. True disappearance vs window-boundary dropout needs to be distinguished by checking the post dates of the "missing" posts.

Similarly, "Emerging" topics in Run 3 can be found by checking which Run 3 topic IDs never appear as "New Topic" in the evolution DataFrame.

QFU-9: **[HARD]** What is the Jaccard upper bound trick and how would you implement it? (Follow-up to Q13)

Ideal Answer:

The Jaccard upper bound trick allows you to skip computing the exact Jaccard score for a pair when you can prove in advance that it cannot exceed a threshold.

The bound: Jaccard(A, B) ≤ min(|A|, |B|) / max(|A|, |B|). This is because the best possible case is when the smaller set is fully contained in the larger set, giving intersection = min(|A|,|B|) and union = max(|A|,|B|).

Implementation:

- Before computing intersection, check: if min(|A|,|B|) / max(|A|,|B|) < threshold (e.g., 0.1), skip this pair entirely — it cannot qualify as anything above Dropped/Noise.
- This is especially powerful when topic sizes are very asymmetric — a topic with 5 posts can never have Jaccard > 0.1 against a topic with 100 posts.

In Python: `if min(len_a, len_b) / max(len_a, len_b) < min_threshold: continue`. This single line can eliminate 60–80% of pair computations in typical topic size distributions, with zero loss of result quality above the threshold.

QFU-10: **[MEDIUM]** After fixing the TF-IDF vectorizer bug, what would the correct interpretation of cosine similarity be that you could not claim before? (Follow-up to Q14)

Ideal Answer:

Before the fix: each pair's cosine score is computed in its own private vocabulary of 2 documents. The IDF values are meaningless (log(2/1) = 0.69 for all terms in both docs, log(2/2) = 0 for shared terms). The vocabulary changes per pair. Scores are not comparable across pairs.

After fitting once on the full corpus of all topic texts:

- IDF weights are meaningful: common words across many topics (e.g., "news", "reported") get low IDF; rare, discriminating words (e.g., "Punxsutawney", "filibuster") get high IDF.
- Scores are comparable: a cosine of 0.7 between Topic A and Topic B means the same thing as 0.7 between Topic C and Topic D — both measured on the same vector space.
- You can now rank all pairs globally by cosine score and say "these are the top 10 most textually similar topic pairs across the entire dataset" — a statement impossible to make with per-pair fitting.

The fix also opens the door to computing the full T1×T2 cosine matrix in one operation using matrix multiplication — a dramatic performance improvement.

QFU-11: **[MEDIUM]** How would you add data validation and schema checking at the point of ingestion? (Follow-up to Q16)

Ideal Answer:

I would implement a validation layer using Pydantic models or a lightweight custom schema checker before any processing begins. The structure:

- Define a PostSchema: id (str, required, non-empty), topic (numeric or -1), entities (list or None), post_date (YYYY-MM-DD format).
- Define a TopicSchema: topic (numeric), title (str), summary (list[str] or str), key_entities (list[list[str]]).

- On load, validate every record against its schema. Collect all validation errors rather than failing fast — you want a full error report, not just the first broken record.
- Emit a validation summary: X posts failed schema (Y%), Z topics failed schema. If failure rate exceeds a threshold, halt the pipeline with a clear error.

Additional checks beyond schema:

- Referential integrity: every topic ID in posts must have a corresponding entry in topic_entities.
- ID uniqueness: assert no duplicate post IDs within a single file.
- Date range: post_date should fall within the declared data window for that file.

QFU-12: **[MEDIUM]** What metrics would you use to measure the quality of the topic evolution categorization beyond accuracy? (Follow-up to Q17)

✅ Ideal Answer:

Accuracy is a poor metric here because classes are heavily imbalanced — "Stable" and "Dropped/Noise" are likely the most common, while "Evolved" and "Fragmented" are rarer and arguably more important to get right.

Better metrics:

- Per-class F1: Separately measure F1 for Stable, Evolved, Fragmented, and Dropped/Noise. A poor F1 on Fragmented (the most nuanced category) is more damning than high overall accuracy.
- Cohen's Kappa: Measures inter-rater agreement corrected for chance — important if multiple annotators label the ground truth.
- Confusion matrix analysis: Which categories get confused most? Stable vs Evolved confusion is forgivable; Stable vs Dropped/Noise confusion is a serious error.
- Threshold calibration curve: Plot F1 vs Jaccard threshold for each category. The "elbow" in this curve is the optimal threshold — not a gut-feel number.
- Downstream task metric: Ultimately, measure whether analysts using the pipeline make better or faster decisions. Human evaluation is the gold standard for a tool designed for human consumption.

PART 2 FOLLOW-UPS

QFU-13: **[MEDIUM]** How would you evaluate whether the multi-qa model selection was actually correct for this use case? (Follow-up to Q18)

✅ Ideal Answer:

Model selection should be empirically validated, not just justified by theory. Evaluation approach:

- Create a benchmark query set: write 20–30 realistic user queries covering different intent types (entity-specific, theme-based, event-based, emotional/sentiment queries).
- For each query, manually annotate the top-10 relevant topics as a gold standard (binary relevant/not-relevant).
- Run the same queries through both all-MiniLM-L6-v2 and multi-qa-MiniLM-L6-cos-v1, returning top-5 results each.
- Compute nDCG@5 (normalized Discounted Cumulative Gain) and MRR (Mean Reciprocal Rank) for each model across all queries.

The model with higher nDCG@5 is empirically better for this retrieval task. If the difference is small (< 2–3 points), the choice matters little and the simpler/faster model should be preferred. If there are specific query types where one model dominates, document those as known strengths/weaknesses.

QFU-14: **[MEDIUM]** What would a Jaccard threshold of 0.1 do to your search results? What about 0.95? (Follow-up to Q19)

✅ Ideal Answer:

Jaccard threshold of 0.1 (very permissive deduplication):

- Almost no topics are deduplicated — two topics sharing just 10% of their word tokens are kept as separate entries.
- The embedding index grows much larger, closer to the full combined topic count across all three runs.
- Search results become noisier — near-duplicate topics from different runs frequently appear in the top-5, reducing diversity and making results feel repetitive.

Jaccard threshold of 0.95 (very aggressive deduplication):

- Only near-identical topics are merged — the index becomes very compact.
- Risk of false merges: distinct topics that share a common domain vocabulary (e.g., two different "Elon Musk" topics) could be incorrectly collapsed into one, making one of them permanently unsearchable.
- Search results become sparse in heavily-covered topic areas — important nuances may be lost.

The sweet spot depends on the vocabulary richness of the domain. For news topics with rich, specific vocabulary, 0.4–0.6 is typically appropriate. Always tune against a held-out evaluation set.

QFU-15: **[HARD]** How would you handle a user query that contains negation — e.g., "I want protest topics but NOT violent ones"? (Follow-up to Q20)

✅ Ideal Answer:

Standard cosine similarity with embeddings handles negation poorly because "not violent protests" and "violent protests" produce similar embeddings — the semantic space is not oriented around logical negation.

Three strategies in increasing effectiveness:

- Query decomposition: Parse the user query into a positive component ("protest topics") and a negative component ("violent"). Embed both separately. Compute cosine similarity for positive minus cosine similarity for negative, with a tunable weight. Topics that score high on positive and low on negative are ranked first.
- Re-ranking with LLM: Retrieve top-20 results using pure semantic search, then use a cheap LLM call to re-rank them by applying the negation filter. This avoids the "check every topic" constraint — you still use embedding for the candidate set.
- Hard exclusion with keyword detection: If the negative clause is entity-specific ("NOT Kendrick Lamar"), use exact string matching to post-filter results. Cheap and effective for named entity negations.

The first strategy is purely embedding-based and scalable. The second is a hybrid that respects the spirit of the constraint (no full-corpus LLM scan). For production, combine both.

QFU-16: **[MEDIUM]** How would you handle topics with very long summaries that exceed the model's 128-token limit? (Follow-up to Q21)

✅ Ideal Answer:

MiniLM has a 128-token maximum. A long summary + title + entities combination will be silently truncated, meaning the tail of the summary is dropped before embedding — you lose information from the end of the text without any warning.

Mitigation strategies:

- Prioritize ordering: Put the title first, then key entities, then summary. The most discriminating information (title + entities) is preserved even if the summary is truncated. This is an easy one-line fix.
- Summarize the summary: Run a short extractive summarization (e.g., take the first 2 bullet points) to cap the summary contribution at ~50 tokens. This is deterministic and cheap.
- Use a longer-context model: Switch to a model with 256 or 512 token context (e.g., all-mpnet-base-v2 with 384 tokens). Slightly more compute per embedding but solves the truncation problem.
- Chunking + pooling: Split long texts into 128-token chunks, embed each chunk, and average the embeddings. This is a common production technique for long-document retrieval.

QFU-17: **[HARD]** How would you implement a hybrid BM25 + semantic embedding system for this use case? (Follow-up to Q22)

✅ **Ideal Answer:**

A hybrid system runs both retrieval methods in parallel and fuses their rankings. Here is the concrete architecture:

Offline indexing: Build a BM25 index (using rank_bm25 library) over the combination strings. Separately build the FAISS vector index of sentence embeddings. Both can be precomputed and cached.

Online query: (1) BM25 retrieves top-K_1 topics by keyword relevance. (2) Embedding retrieval fetches top-K_2 topics by semantic similarity. Use $K_1 = K_2 = 50$ to give the fusion step enough candidates.

Rank fusion: Apply Reciprocal Rank Fusion (RRF): for each topic, RRF_score = Σ 1/(rank_i + 60) across all ranking lists it appears in. This is parameter-free, robust, and consistently outperforms linear score combination in benchmarks.

Optional re-ranking: Pass top-10 RRF results through a cross-encoder (a more expensive but accurate model) for final re-ranking. Cross-encoders compare query and document jointly, handling nuanced relevance that bi-encoders miss.

This approach combines BM25's precision on exact entity matches with semantic search's strength on paraphrase matching — the best of both worlds.

QFU-18: **[HARD]** What is HyDE and how would you implement it as an enhancement to the search system? (Follow-up to Q24)

✅ **Ideal Answer:**

HyDE (Hypothetical Document Embeddings) is a technique where instead of embedding the user's query directly, you use an LLM to generate a hypothetical document that would answer the query, then embed that document.

The insight: a user query like "I want protest topics with emotional language" is short and structurally very different from a topic combination string (title + summary + entities). Embedding them in the same space works reasonably well, but a hypothetical topic description is structurally identical to real topic descriptions — the embedding match is much tighter.

Implementation:

- User submits query: "I want topics about recent violence during protests with emotional content."
- LLM call (single, cheap): "Generate a fake topic description in the format Title: ...; Summary: ...; entities: ... that would match this query."
- Embed the LLM-generated hypothetical description (not the original query).

- Use that embedding for cosine retrieval against the real topic index.

This respects the "no LLM to check every topic" constraint — you use the LLM once for query expansion, not T times. In practice, HyDE improves recall significantly for abstract or intent-heavy queries.

QFU-19: **[HARD]** How would you handle a situation where the embedding model needs to be swapped out? What is the migration path? (Follow-up to Q26)

✅ Ideal Answer:

Swapping embedding models is a breaking change because the new model produces vectors in a different geometric space — cosine similarity between an old embedding and a new embedding is meaningless.

Migration path:

- Step 1 — Parallel indexing: Deploy the new model alongside the old one. Re-embed the full topic corpus with the new model and store it in a new index. Do not delete the old index yet.
- Step 2 — Shadow mode: Route production queries through both models. Log results from both but serve only old model results to users. Compare quality offline.
- Step 3 — A/B test: Gradually shift traffic to the new model (10% → 50% → 100%). Monitor nDCG, user click-through rates, and latency.
- Step 4 — Cutover: Once the new model is validated, retire the old index. Going forward, all new topics are indexed with the new model.

Key engineering requirement: decouple the embedding model version from the index. Store a model_version tag alongside every embedding in the vector DB. This allows multi-model indexes and makes rollback trivial — just switch which version queries are routed to.

QFU-20: **[MEDIUM]** What is the first data quality check you would add before any processing begins? (Follow-up to Q27)

✅ Ideal Answer:

The single most impactful first check: referential integrity between posts and topics.

Specifically: every topic ID that appears in the posts array must have a corresponding entry in the topic_entities array (excluding -1). If a post references topic 42 but there is no topic 42 in topic_entities, every downstream metric for that topic will be wrong — it will have posts assigned to it but no title, summary, or entities to match against.

Implementation (5 lines of Python):

- post_topics = set of unique topic IDs from posts array (excluding -1).
- meta_topics = set of topic IDs from topic_entities array.
- orphaned = post_topics - meta_topics: topics referenced by posts but with no metadata.
- If len(orphaned) > 0: log a warning with the count and a sample of orphaned IDs.

This check catches the most dangerous class of data errors — silent silent corruption where metrics look plausible but are computed on incomplete data. It is cheap (set operations), fast, and directly actionable.

QFU-21: **[MEDIUM]** If you had one week to productionize just one of the two parts, which would you choose and why? (Follow-up to Q29)

✅ Ideal Answer:

I would choose Part 2 — the semantic search system.

Three reasons:

- User-facing value: Search has a direct, immediate user interface. A working search box delivers tangible value from day one. Part 1's topic evolution analysis is valuable but primarily for analysts doing periodic review — not an interactive, daily-use feature.
- Productionization complexity: Part 2 has a cleaner production path — the core components (embedding model, vector index, search API endpoint) are all well-understood infrastructure. Part 1 would require resolving fundamental algorithmic issues (topic -1 handling, the TF-IDF vectorizer bug, split/merge detection) before it is reliable enough to deploy.
- Validation is easier: Search quality can be measured quickly with click-through rate and user feedback. Topic evolution quality requires expensive annotation campaigns to validate properly.

The one-week plan for Part 2: Day 1–2 fix the deduplication and add HyDE. Day 3 set up a vector DB (pgvector in Postgres). Day 4 build a simple FastAPI endpoint with query → embed → search → return JSON. Day 5 write evaluation queries and measure nDCG@5, iterate on the combination string format. Day 6–7 add logging, basic monitoring, and a simple frontend.

— End of Interview Preparation Guide —